

KONGU ENGINEERING COLLEGE

IOT-BASED WORKER HEALTH AND SAFETY MONITORING SYSTEM

MECHATRONICS AND IOT

EXCERSICE – 05

HEMAVARSHINI S - 24MER017

KAVIN KUMARAN S - 24MER023

KOWSIKA K - 24MER026

VISHNUPRASATH B - 24MER062

IOT-BASED WORKER HEALTH AND SAFETY MONITORING SYSTEM

1. Project Overview

Introduction

Workers in construction sites, industries, mines, and other hazardous environments are often exposed to unsafe conditions such as high temperature, high humidity, harmful gases, and physical stress. Continuous manual monitoring of every worker under such conditions is impractical, and unsafe conditions can go unnoticed until an incident occurs.

The IoT-Based Worker Health and Safety Monitoring System is an ESP32-based safety system designed to continuously monitor the physiological and environmental condition of a worker. The system acquires temperature, humidity, gas concentration, heart rate, and GPS location in real time, displays the readings through the Serial Monitor, and uploads the data to the ThingSpeak IoT cloud platform over Wi-Fi.

A buzzer is incorporated to provide an immediate local alert whenever any monitored parameter crosses its programmed safety limit, enabling both the worker and remote supervisors to respond quickly to unsafe conditions.

Main Features

- Continuous monitoring of surrounding temperature and humidity.
- Harmful/combustible gas detection.
- Real-time heart-rate (BPM) monitoring.
- GPS-based worker location tracking.
- Automatic threshold-based unsafe-condition detection.
- Immediate local buzzer alert.
- Real-time display through the Serial Monitor.
- Cloud data logging and remote monitoring through ThingSpeak.
- Wi-Fi based IoT connectivity using ESP32.

2. Problem Statement

Workers in construction sites, industries, mines, and other hazardous environments may be exposed to high temperature, high humidity, harmful gases, physical stress, and generally unsafe working conditions.

Manual monitoring cannot continuously track these parameters for every worker, which can result in:

- Delayed detection of unsafe environmental conditions.
- Undetected exposure to harmful or combustible gases.
- Unnoticed physiological stress, such as an abnormally high heart rate.
- Difficulty locating a worker during an emergency.
- Delayed emergency response.
- Increased risk of workplace accidents and injuries.

Therefore, an automated system is required to continuously monitor worker health and environmental parameters and provide an immediate alert together with location information when unsafe conditions are detected.

3. Proposed Solution

The proposed system uses multiple sensors and a microcontroller to automate worker safety monitoring. The DHT11 sensor measures ambient temperature and humidity, the MQ-2 sensor detects harmful or combustible gases, the pulse sensor measures the worker's heart rate, and the GPS module tracks the worker's location. All sensor outputs are sent to the ESP32.

The ESP32 processes these values and evaluates them against the programmed safety limits to determine the worker's safety status.

When any parameter goes outside its safe range:

Sensor (Temp / Humidity / Gas / BPM) → ESP32 → Safety = UNSAFE → Buzzer ON

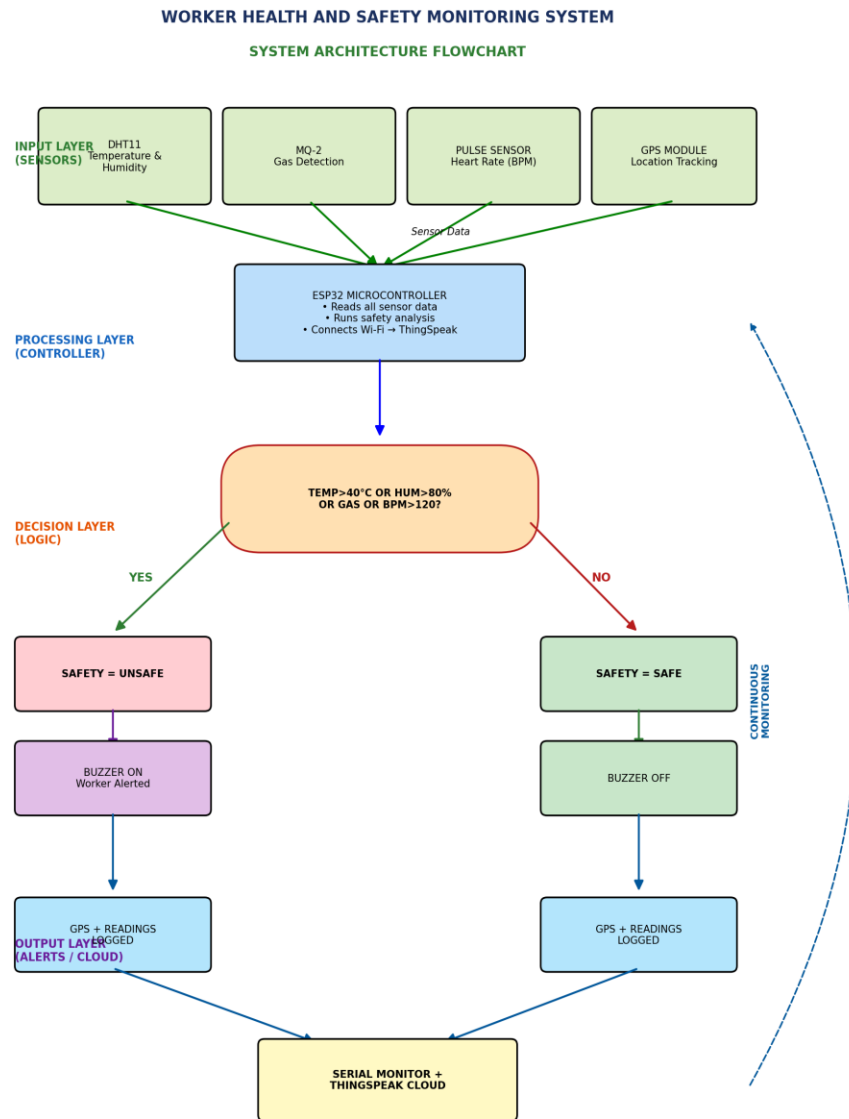
When all parameters remain within the safe range:

Sensor (Temp / Humidity / Gas / BPM) → ESP32 → Safety = SAFE → Buzzer OFF

In parallel with the local alarm logic, the ESP32 continuously displays the readings and GPS location on the Serial Monitor and uploads them to the ThingSpeak cloud platform, enabling both immediate local warning and remote monitoring of worker safety and location.

4. System Architecture

The overall architecture of the system is:



System Flow:

Sensors (DHT11, MQ-2, Pulse, GPS) → ESP32 → Safety Analysis → Buzzer / Serial Monitor / ThingSpeak

5. Circuit Table

Component	Pin	ESP32
DHT11 Sensor	DATA	GPIO 4
DHT11 Sensor	VCC / GND	3V3 / GND
MQ-2 Gas Sensor	DO	GPIO 27
MQ-2 Gas Sensor	VCC / GND	5V / GND
Pulse Sensor	AO	GPIO 34
Pulse Sensor	VCC / GND	3V3 / GND
Buzzer	+	GPIO 26
Buzzer	–	GND
GPS Module	TX	GPIO 16 (RX2)
GPS Module	RX	GPIO 17 (TX2)
GPS Module	VCC / GND	3V3 / GND
All Sensors	GND	ESP32 GND (common)

GPS Connection

The GPS communication is crossed and uses ESP32's Hardware Serial 2:

GPS TX → ESP32 GPIO 16 (RX2)

GPS RX → ESP32 GPIO 17 (TX2)

Power Supply

The ESP32 can be powered through USB or a regulated 5V supply. The DHT11, pulse sensor and GPS module operate from the 3V3 rail, while the MQ-2 sensor typically requires a 5V supply. All sensor ground pins must share a common ground with the ESP32.

6. Circuit Design

The circuit consists of five major sections:

6.1 ESP32 Microcontroller

The ESP32 is the main controller. It reads all sensor signals, evaluates the worker's safety status, drives the buzzer, and connects to Wi-Fi to upload data to ThingSpeak.

6.2 DHT11 Sensor

The DHT11 measures ambient temperature and humidity and is connected to a digital data pin.

DHT11 DATA → ESP32 GPIO 4

6.3 MQ-2 Gas Sensor

The MQ-2 provides a digital output indicating the presence of harmful or combustible gas.

MQ-2 DO → ESP32 GPIO 27

6.4 Pulse Sensor

The pulse sensor provides an analog signal used to calculate the worker's heart rate (BPM).

Pulse Sensor A0 → ESP32 GPIO 34

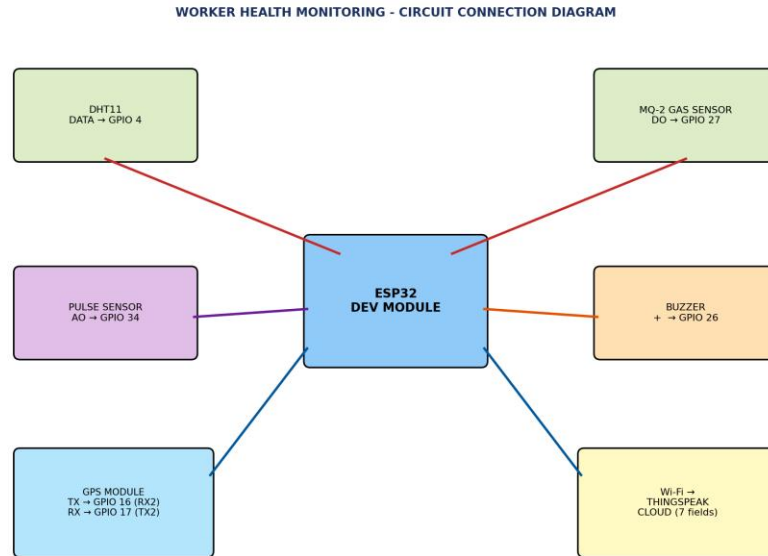
6.5 GPS Module and Buzzer

The GPS module communicates with the ESP32 over Hardware Serial 2 to provide location data, while the buzzer provides the local audible alert.

GPS TX/RX → ESP32 GPIO 16/17

ESP32 GPIO 26 → Buzzer

CIRCUIT DESIGN:



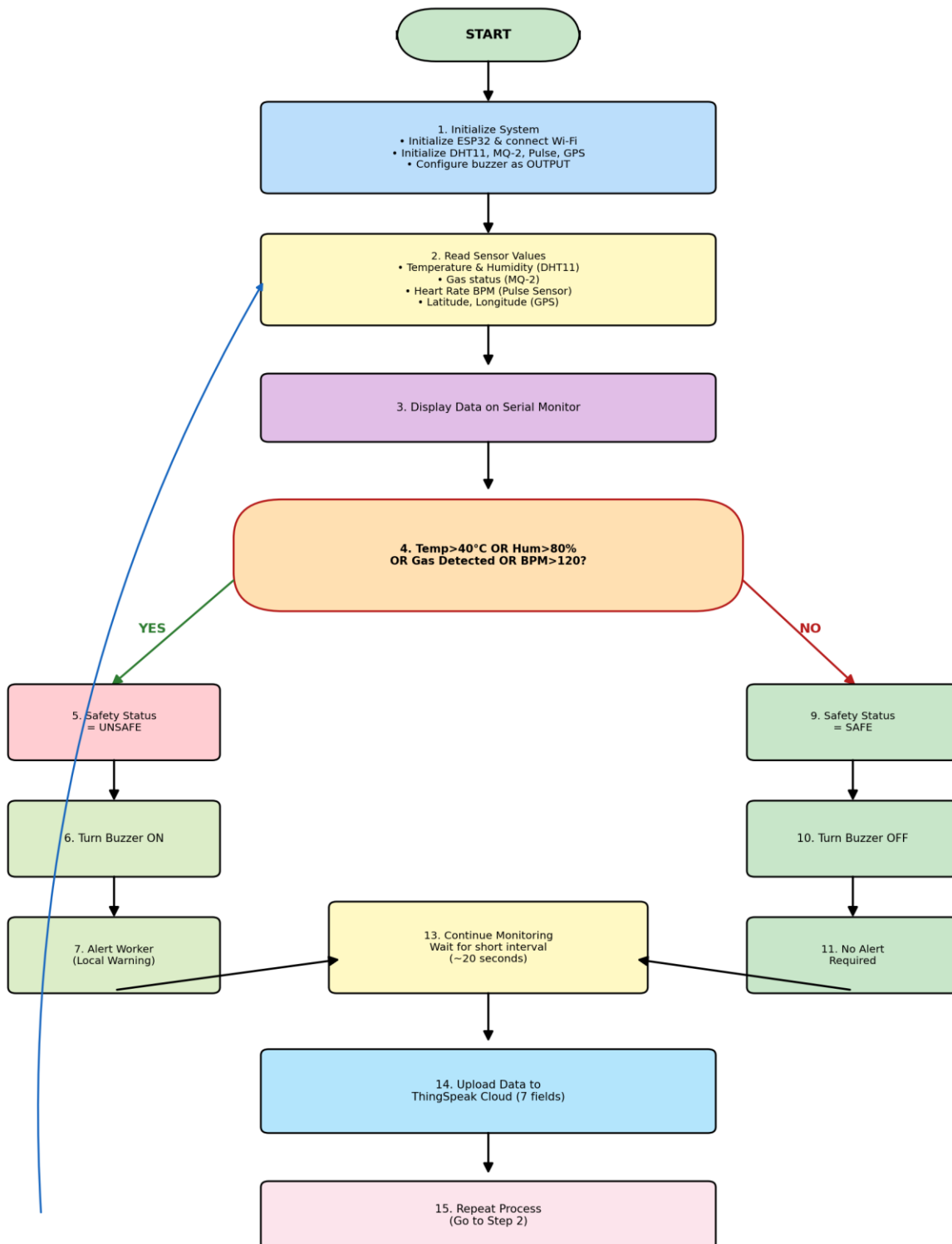
7. Algorithm

Algorithm

1. Start the system.
2. Initialize the ESP32 and connect to Wi-Fi.
3. Initialize the DHT11, MQ-2, pulse sensor and GPS module.
4. Configure the buzzer pin as an output.
5. Read temperature and humidity from the DHT11.
6. Read the gas status from the MQ-2.
7. Read the pulse sensor and calculate heart rate (BPM).
8. Read the GPS module for latitude, longitude and fix status.
9. Display all readings on the Serial Monitor.
10. Check whether temperature > 40°C, humidity > 80%, gas is detected, or BPM > 120.
11. If any condition is true, set Safety Status = UNSAFE and turn ON the buzzer.
12. If no condition is true, set Safety Status = SAFE and turn OFF the buzzer.
13. Upload temperature, humidity, heart rate, gas status, GPS coordinates and safety status to ThingSpeak.
14. Wait for approximately 20 seconds.
15. Repeat the process continuously.

WORKER HEALTH MONITORING SYSTEM

ALGORITHM FLOWCHART



8. Coding

The following Arduino program is designed for the ESP32 used in the project.

```
#include <WiFi.h>
#include <HTTPClient.h>
#include <DHT.h>
#include <TinyGPS++.h>

// =====
// WIFI SETTINGS
// =====

const char* ssid = "kowsi";
const char* password = "kowsi @ 01";

// ThingSpeak Write API Key
String apiKey = "20QR1SF2BOGLN7I0";

// =====
// PIN DEFINITIONS
// =====

#define DHTPIN    4
#define DHTTYPE   DHT11

#define MQ2_PIN   27
#define BUZZER_PIN 26
#define PULSE_PIN 34

// GPS
#define GPS_RX    16 // ESP32 RX <- GPS TX
#define GPS_TX    17 // ESP32 TX -> GPS RX

// =====
// OBJECTS
// =====
```

```
DHT dht(DHTPIN, DHTTYPE);
```

```
TinyGPSPlus gps;
```

```
HardwareSerial gpsSerial(2);
```

```
// =====  
// VARIABLES  
// =====
```

```
float temperature = 0;
```

```
float humidity = 0;
```

```
int pulseValue = 0;
```

```
int bpm = 0;
```

```
bool beatDetected = false;
```

```
unsigned long lastBeat = 0;
```

```
bool gasDetected = false;
```

```
bool unsafe = false;
```

```
// =====  
// THINGSPEAK TIMER  
// =====
```

```
unsigned long lastThingSpeak = 0;
```

```
const unsigned long thingSpeakInterval = 20000;
```

```
// =====  
// SETUP  
// =====
```

```
void setup()
```

```
{
  Serial.begin(115200);

  delay(1000);

  Serial.println();
  Serial.println("=====");
  Serial.println("  WORKER SAFETY MONITORING SYSTEM");
  Serial.println("=====");

  // =====
  // GPS
  // =====

  gpsSerial.begin(
    9600,
    SERIAL_8N1,
    GPS_RX,
    GPS_TX
  );

  Serial.println("GPS Serial Started");

  // =====
  // SENSOR PINS
  // =====

  pinMode(MQ2_PIN, INPUT);

  pinMode(BUZZER_PIN, OUTPUT);

  digitalWrite(BUZZER_PIN, LOW);

  // =====
  // DHT11
  // =====
```

```
dht.begin();

Serial.println("DHT11 Started");

// =====

// WIFI

// =====

Serial.println();
Serial.print("Connecting to WiFi: ");
Serial.println(ssid);

WiFi.begin(
  ssid,
  password
);

while (WiFi.status() != WL_CONNECTED)
{
  delay(500);
  Serial.print(".");
}

Serial.println();
Serial.println("WiFi Connected!");

Serial.print("ESP32 IP Address: ");
Serial.println(WiFi.localIP());

Serial.println();
Serial.println("System Ready");
Serial.println("=====");
}

// =====
```

```
// LOOP
// =====

void loop()
{
  // =====
  // DHT11
  // =====

  float newTemperature =
    dht.readTemperature();

  float newHumidity =
    dht.readHumidity();

  if (!isnan(newTemperature) &&
      !isnan(newHumidity))
  {
    temperature =
      newTemperature;

    humidity =
      newHumidity;
  }

  // =====
  // MQ-2
  // =====

  gasDetected =
    (digitalRead(MQ2_PIN) == LOW);

  // =====
  // PULSE SENSOR
  // =====
```

```
pulseValue =  
  analogRead(PULSE_PIN);  
  
unsigned long currentTime =  
  millis();  
  
// Simple pulse detection  
// ESP32 ADC = approximately 0 to 4095  
// Adjust threshold according to your sensor.  
  
if (pulseValue > 2000 &&  
    !beatDetected)  
{  
  beatDetected = true;  
  
  if (lastBeat > 0)  
  {  
    unsigned long interval =  
      currentTime - lastBeat;  
  
    if (interval > 300 &&  
        interval < 2000)  
    {  
      bpm =  
        60000 / interval;  
    }  
  }  
  
  lastBeat =  
    currentTime;  
}  
  
if (pulseValue < 1800)  
{  
  beatDetected = false;  
}
```

```
// =====  
// GPS  
// =====  
  
while (gpsSerial.available())  
{  
    gps.encode(  
        gpsSerial.read()  
    );  
}  
  
// =====  
// SAFETY CHECK  
// =====  
  
unsafe = false;  
  
// Temperature  
if (temperature > 40)  
{  
    unsafe = true;  
}  
  
// Humidity  
if (humidity > 80)  
{  
    unsafe = true;  
}  
  
// Gas  
if (gasDetected)  
{  
    unsafe = true;  
}
```

```
// Heart rate
if (bpm > 120)
{
    unsafe = true;
}

// =====

// BUZZER
// =====

if (unsafe)
{
    digitalWrite(
        BUZZER_PIN,
        HIGH
    );
}
else
{
    digitalWrite(
        BUZZER_PIN,
        LOW
    );
}

// =====

// SERIAL MONITOR
// =====

Serial.println();
Serial.println("-----");

Serial.print("Temperature : ");
Serial.print(temperature, 1);
Serial.println(" °C");
```



```
Serial.print("Humidity  : ");  
Serial.print(humidity, 1);  
Serial.println(" %");
```

```
Serial.print("Heart Rate : ");  
Serial.print(bpm);  
Serial.println(" BPM");
```

```
Serial.print("Pulse ADC  : ");  
Serial.println(pulseValue);
```

```
Serial.print("Gas Status : ");
```

```
if (gasDetected)  
{  
    Serial.println("DANGER");  
}  
else  
{  
    Serial.println("SAFE");  
}
```

```
// =====  
// GPS OUTPUT  
// =====
```

```
if (gps.location.isValid())  
{  
    Serial.print("GPS Status : FIXED");  
    Serial.println();  
  
    Serial.print("Latitude  : ");  
    Serial.println(  
        gps.location.lat(),  
        6  
    );
```

```
Serial.print("Longitude  : ");
Serial.println(
  gps.location.lng(),
  6
);
}
else
{
  Serial.println(
    "GPS Status : SEARCHING..."
  );
}
```

```
// =====
// SATELLITES
// =====
```

```
if (gps.satellites.isValid())
{
  Serial.print("Satellites : ");
  Serial.println(
    gps.satellites.value()
  );
}
```

```
// =====
// SAFETY STATUS
// =====
```

```
Serial.print("Safety    : ");
```

```
if (unsafe)
{
  Serial.println("!!! UNSAFE !!!");
}
```

```
else
{
  Serial.println("SAFE");
}

// =====
// WIFI STATUS
// =====

Serial.print("WiFi    : ");

if (WiFi.status() == WL_CONNECTED)
{
  Serial.println("CONNECTED");
}
else
{
  Serial.println("DISCONNECTED");
}

Serial.println("-----");

// =====
// THINGSPEAK UPDATE
// =====

if (millis() - lastThingSpeak >=
    thingSpeakInterval)
{
  lastThingSpeak =
    millis();

  sendToThingSpeak();
}

delay(1000);
```

```

}

// =====
// THINGSPEAK FUNCTION
// =====

void sendToThingSpeak()
{
  Serial.println();
  Serial.println(">>> Sending data to ThingSpeak...");

  if (WiFi.status() != WL_CONNECTED)
  {
    Serial.println(
      "ERROR: WiFi disconnected"
    );

    return;
  }

  HTTPClient http;

  String url =
    "http://api.thingspeak.com/update?api_key=";

  url += apiKey;

  // =====
  // FIELD 1 - TEMPERATURE
  // =====

  url += "&field1=";
  url += String(
    temperature,
    1
  );

```

```
// =====  
// FIELD 2 - HUMIDITY  
// =====
```

```
url += "&field2=";  
url += String(  
    humidity,  
    1  
);
```

```
// =====  
// FIELD 3 - HEART RATE  
// =====
```

```
url += "&field3=";  
url += String(  
    bpm  
);
```

```
// =====  
// FIELD 4 - GAS  
// =====
```

```
url += "&field4=";
```

```
if (gasDetected)  
{  
    url += "1";  
}  
else  
{  
    url += "0";  
}
```

```
// =====
```

```
// FIELD 5 - LATITUDE
```

```
// =====
```

```
if (gps.location.isValid())
```

```
{
```

```
    url += "&field5=";
```

```
    url += String(
```

```
        gps.location.lat(),
```

```
        6
```

```
    );
```

```
// =====
```

```
// FIELD 6 - LONGITUDE
```

```
// =====
```

```
url += "&field6=";
```

```
url += String(
```

```
    gps.location.lng(),
```

```
    6
```

```
);
```

```
}
```

```
// =====
```

```
// FIELD 7 - SAFETY
```

```
// =====
```

```
url += "&field7=";
```

```
if (unsafe)
```

```
{
```

```
    url += "1";
```

```
}
```

```
else
```

```
{
```

```
    url += "0";
}

// =====
// SEND DATA
// =====

http.begin(url);

int httpCode =
    http.GET();

if (httpCode > 0)
{
    Serial.print(
        "ThingSpeak HTTP Code: "
    );

    Serial.println(
        httpCode
    );

    String response =
        http.getString();

    Serial.print(
        "ThingSpeak Entry ID: "
    );

    Serial.println(
        response
    );
}
else
{
    Serial.print(
```

```

    "ThingSpeak ERROR: "
);

Serial.println(
    httpCode
);
}

http.end();

Serial.println(
    ">>> ThingSpeak update complete"
);
}

```

Code Explanation

- GPIO 4 receives data from the DHT11 (temperature and humidity).
- GPIO 27 reads the digital output of the MQ-2 gas sensor.
- GPIO 34 reads the analog pulse sensor used to calculate BPM.
- GPIO 26 controls the buzzer.
- GPIO 16/17 form Hardware Serial 2, used for GPS communication via TinyGPSPlus.
- tempLimit, humLimit and bpmLimit define the programmed safety thresholds.
- ThingSpeak.setField() and ThingSpeak.writeFields() upload all seven parameters to the cloud channel.

The pulse-detection logic in this prototype is a simple threshold-based method and should not be treated as a medical-grade heart-rate measurement. The threshold values should be adjusted according to the actual sensors and site safety requirements.

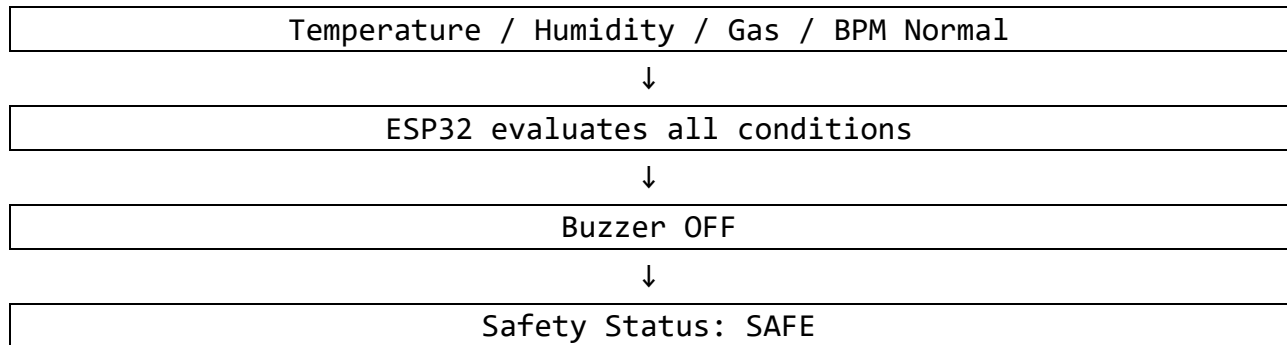
9. System Working

When the system is powered ON, the ESP32 initializes all connected sensors, configures the buzzer as an output, and connects to the Wi-Fi network.

The DHT11, MQ-2, pulse sensor and GPS module continuously provide readings of the worker's physiological state, the surrounding environment, and the worker's location.

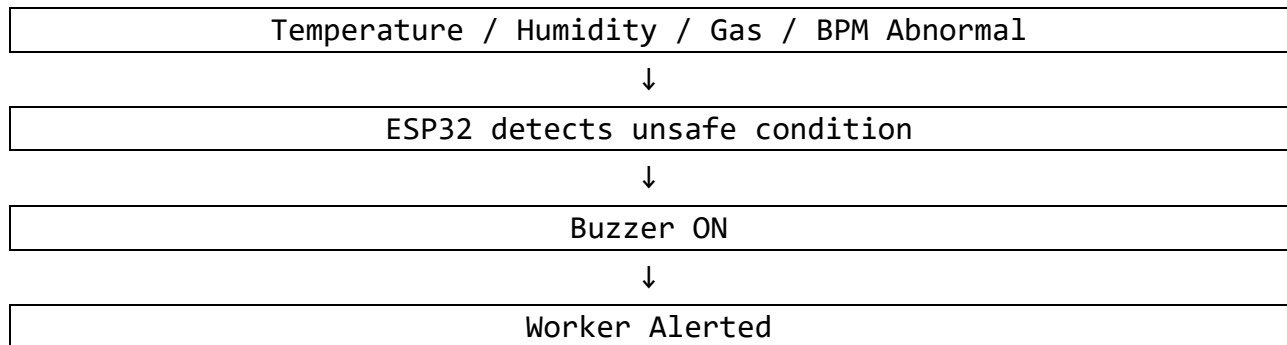
Safe Condition

When all measured parameters are within their programmed safe range:

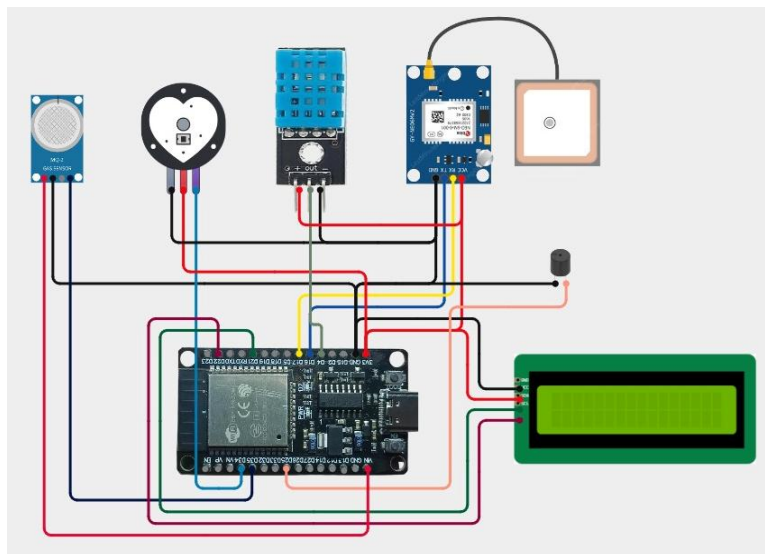


Unsafe Condition

When any parameter crosses its threshold:



At the same time, the ESP32 uploads the current readings, GPS location, and safety status to ThingSpeak, allowing supervisors to remotely monitor the worker's condition and location. This process repeats automatically, allowing the system to maintain continuous worker safety monitoring.



10. Bill of Materials

S.No.	Component	Quantity
1	ESP32 Development Board	1
2	DHT11 Sensor	1
3	MQ-2 Gas Sensor	1
4	Pulse Sensor	1
5	GPS Module	1
6	Buzzer	1
7	Breadboard	1
8	Jumper Wires	As required
9	Regulated Power Supply	1

Software Requirements

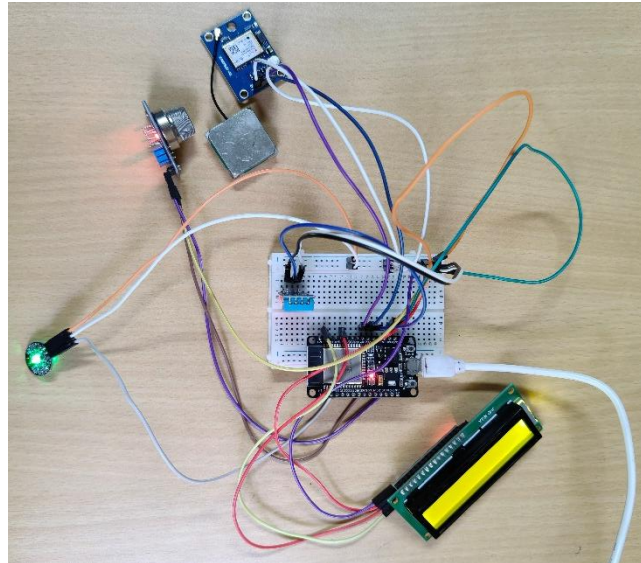
- Arduino IDE
- ESP32 Board Package
- DHT Library
- Adafruit Unified Sensor
- TinyGPSPlus
- WiFi.h and HTTPClient.h
- ThingSpeak Arduino Library

11. Prototype Implementation

The prototype was assembled using an ESP32 development board, a DHT11 sensor, an MQ-2 gas sensor, a pulse sensor, a GPS module, and a buzzer, wired on a breadboard as per the circuit table above.

The ESP32 was selected as the main controller because it provides sufficient analog and digital I/O, an additional hardware serial port for the GPS module, and built-in Wi-Fi connectivity needed for direct IoT cloud integration. Since an OLED/LCD was removed from the system, the Serial Monitor is used for real-time local output.

The prototype continuously reads temperature, humidity, gas status, heart rate and GPS location, evaluates the worker's safety status, drives the buzzer accordingly, and streams the readings to the Serial Monitor and the ThingSpeak channel.



12. Results & Performance Analysis

Expected Results

The system successfully demonstrates continuous worker health and safety monitoring with automatic alerting and location tracking.

Condition	Sensor Condition	Buzzer	Safety Status
Normal condition	All parameters within limits	OFF	SAFE
High temperature / humidity	Temp>40°C or Hum>80%	ON	UNSAFE
Gas detected	MQ-2 output = DANGER	ON	UNSAFE
Abnormal heart rate	BPM > 120	ON	UNSAFE

Performance

The system provides the following benefits:

- Real-time worker monitoring.
- Automatic unsafe-condition detection.
- Immediate local buzzer alert.
- GPS-based location tracking for emergency response.
- Wireless IoT monitoring through ESP32.
- Cloud-based data visualization through ThingSpeak.
- Low-cost, expandable prototype suitable for industrial safety applications.

Applications

- Construction sites
- Manufacturing industries
- Mining areas
- Chemical industries
- Warehouses
- Fire-risk environments
- Hazardous industrial areas
- Remote work locations

Conclusion

The IoT-Based Worker Health and Safety Monitoring System provides an integrated approach to worker safety by combining ESP32, DHT11, MQ-2, a pulse sensor, GPS, a buzzer, Wi-Fi, and ThingSpeak. The system continuously monitors the worker's environmental and physiological parameters and reports SAFE when all values remain within the programmed limits.

If a dangerous gas is detected, temperature or humidity exceeds the programmed limit, or the measured heart rate exceeds the programmed threshold, the ESP32 activates the buzzer, reports the condition as UNSAFE, and uploads the reading, along with the GPS location, to ThingSpeak for remote monitoring.

The system can serve as a prototype for smart industrial safety systems, with future improvements such as mobile notifications, improved heart-rate algorithms, additional gas sensors, an emergency button, and a dedicated monitoring dashboard.